

TBD2

AI-Powered Enterprise Risk Intelligence Platform

TECH STACK & ARCHITECTURE RECOMMENDATION

Pre-Seed | Team of 4 | \$2,000/mo Budget | 8-Week MVP

April 2026 | v1.0 | CONFIDENTIAL

Executive Summary

This document is the definitive technology and architecture recommendation for TBD2. Every layer of the stack has been evaluated against three non-negotiable constraints: a bootstrapped infrastructure budget of \$2,000/month, a team of four people with mixed seniority, and an 8-week MVP deadline. Nothing in here is aspirational — every choice is grounded in what this team can build, ship, and operate within those limits.

Bottom Line

The recommended stack is: React + Vite (frontend) · FastAPI (backend) · Celery + Redis (pipeline) · LangChain-lite custom orchestration (LLM layer) · Anthropic Claude API as primary model · PostgreSQL (primary database) · Redis (cache) · Cloudflare R2 (object storage) · Railway or Render for hosting · GitHub Actions for CI/CD · Sentry + Langfuse for observability. This combination fits inside \$1,200–\$1,500/month at MVP, leaves headroom for LLM API costs, and scales cleanly to Series A without a rewrite.

Three decisions remain unresolved and are blocking the API contract freeze on Week 1 Day 3. These are addressed explicitly at the end of this document and must be resolved before any code is written.

1. Frontend Framework

Options Evaluated

Option	Verdict	Notes
React SPA + Vite	 RECOMMENDED	Purpose-built for auth-gated dashboards with complex state. Fastest HMR, smallest bundle, cleanest separation from FastAPI. IIT engineer will be immediately productive.
Next.js	 NOT RECOMMENDED	SSR/SSG benefits are wasted when every page is behind a login wall. Adds Node.js server tier, complicates deployment, and requires managing server-side state the team does not need.
Vue 3 + Vite	 VIABLE ALTERNATIVE	Excellent for dashboards; slightly smaller ecosystem than React for enterprise component libraries. Use only if the frontend engineer has strong Vue preference.
Remix	 NOT RECOMMENDED	Overkill for this use case. Adds complexity without benefit for a fully-authenticated SaaS.
Angular	 NOT RECOMMENDED	Steep learning curve, heavyweight for a 4-person team, slower iteration velocity.

State Management

Option	Verdict	Notes
Zustand	 RECOMMENDED	Minimal boilerplate. Simple enough for an intern to pick up in days. Handles global Co-Pilot panel state, active module context, and pipeline status cleanly.
Redux Toolkit	 VIABLE	Industry standard but more boilerplate. Only justified if the team has existing Redux experience.
React Context + useReducer	 MVP ACCEPTABLE	Acceptable for MVP-only scope. Will create prop-drilling pain as the app grows — migrate to Zustand before Series A.
Jotai / Recoil	 NOT RECOMMENDED	Atomic state model is well-suited to forms but adds mental overhead for pipeline orchestration state.

Component & Chart Libraries

Option	Verdict	Notes
Tailwind CSS (Aegis Quantum tokens)	✓ ALREADY DECIDED	Tailwind is embedded in all HTML prototypes. The Aegis Quantum custom token config must be copy-pasted into tailwind.config.js on Day 1. Non-negotiable.
shadcn/ui	✓ RECOMMENDED for base components	Unstyled headless components that compose cleanly with Tailwind. Gives the IIT engineer accessible, production-grade form inputs, modals, and dropdowns without fighting a third-party design system.
Recharts	✓ RECOMMENDED for charts	Already referenced in PRD. SVG-based, React-first, excellent for SPC charts and sparklines. Handles the 1,200+ metric rendering load well with virtualisation.
D3.js	⚠ SELECTIVE USE ONLY	Use only for custom SPC control charts where Recharts falls short. Full D3 usage will slow down the IIT engineer significantly.
Material UI / Ant Design	✗ NOT RECOMMENDED	Will conflict with Aegis Quantum design tokens. The "no-line" rule and custom surface hierarchy will require overriding virtually every default style — net negative productivity.

Compliance Note

React SPA with a clean FastAPI backend separation is the easiest architecture to audit for SOC 2. All sensitive data stays on the backend; the frontend never holds credentials or raw metric data beyond the session.

API Contract Note

With Vite + React, the frontend engineer can build against MSW (Mock Service Worker) mock endpoints from Day 3 without waiting for a live backend. This is critical given the Week 1 Day 3 contract freeze and the unresolved Q3 (module hierarchy).

2. Backend / API Layer

Framework Choice

Option	Verdict	Notes
FastAPI	 STRONGLY RECOMMENDED	Native async/await for 12 parallel LLM calls. Auto-generates OpenAPI docs (critical for Day 3 contract freeze). Type-safe with Pydantic. Production-proven at Netflix, Uber, Microsoft. Your Python backend developer will be immediately productive.
Django REST Framework	 NOT RECOMMENDED	Synchronous ORM adds latency to every LLM-relayed request. Admin panel advantage is negated by the custom RBAC and audit logging that must be built anyway. Resolving the PRD's ambiguity: FastAPI only.
Flask	 NOT RECOMMENDED	Too low-level. Requires assembling too many libraries for production RBAC, validation, and async support.
Node.js / Express	 NOT RECOMMENDED	Wrong language for a team with Python ML pipelines. Would split the backend codebase language unnecessarily.

API Design: REST vs GraphQL vs tRPC

Option	Verdict	Notes
REST (FastAPI)	 RECOMMENDED	The frontend is a single React app consuming a well-defined set of endpoints. REST is simpler to implement, debug, and document. FastAPI's OpenAPI generation makes the Week 1 Day 3 contract freeze straightforward.
GraphQL	 NOT RECOMMENDED	Over-engineered for this team size. The flexible query model is a benefit when multiple clients (mobile, web, third-party) consume the same API. At MVP with one React frontend, the additional complexity creates more problems than it solves.
tRPC	 NOT RECOMMENDED	Requires TypeScript end-to-end. Your backend developer is Python. Not viable here.

API Gateway

Option	Verdict	Notes
FastAPI built-in routing (MVP)	 RECOMMENDED	No additional gateway needed at MVP. FastAPI handles routing, auth middleware, rate limiting via slowapi, and CORS natively. Saves \$50–200/month vs managed gateways.
AWS API Gateway / Kong	 SERIES A	Introduce at Series A for multi-tenant rate limiting, DDoS protection, and API analytics. Overkill at MVP for a bootstrapped team.
Nginx (reverse proxy)	 RECOMMENDED AT LAUNCH	Nginx as a reverse proxy in front of FastAPI handles SSL termination and static file serving. Zero additional cost.

Real-Time: SSE vs WebSockets

Critical Gap (Not in PRD)

The Pipeline Monitor requires live "64% COMPLETE" progress updates per pipeline. The current stack has no push mechanism — polling every 2 seconds would create unnecessary load. FastAPI natively supports Server-Sent Events (SSE). SSE is the correct choice for one-directional push (server → client): pipeline progress, Co-Pilot streaming responses. WebSockets are only needed for full bidirectional communication, which the MVP does not require. Add two SSE endpoints to the API contract on Day 3: `/pipelines/{id}/stream` and `/copilot/stream`.

3. Task Queue / Async Pipeline

Options Evaluated

Option	Verdict	Notes
Celery + Redis broker	✔ STRONGLY RECOMMENDED	The chord primitive maps exactly to the 12-parallel-module → 1-enterprise barrier pattern. Celery Beat handles scheduled pipeline runs. Redis doubles as the Celery broker and the app cache, reducing infrastructure components. Your Python backend developer almost certainly knows Celery.
ARQ (async Redis queue)	⚠ VIABLE ALTERNATIVE	Lighter than Celery, fully async. Lacks the chord/barrier primitive natively — requires manual coordination for the 12→1 pattern. Only consider if Celery operational overhead is a problem.
Dramatiq	⚠ VIABLE	Simpler API than Celery, supports groups/pipelines. Smaller community and fewer cloud monitoring integrations. Not worth the migration risk from the existing PoC.
RQ (Redis Queue)	✖ NOT RECOMMENDED	Lacks the group/chord pattern needed for the barrier. Would require custom coordination code.
AWS SQS + Lambda	✖ NOT RECOMMENDED	Vendor lock-in. Serverless cold starts are incompatible with the 12 parallel LLM calls that must complete within a bounded window. Too expensive at MVP scale.
Apache Kafka	✖ DEFER	Justified at Series A for real-time streaming ingestion (V2.0 roadmap item). Significantly over-engineered for batch pipeline orchestration at MVP.

Celery Chord Pattern — Implementation Note

The correct implementation for the two-tier LLM pipeline is:

- Create a Celery group of 12 module tasks (each calls its LLM asynchronously)
- Wrap the group in a chord with a single callback task
- The callback is the enterprise-level LLM synthesis call — it fires only after all 12 group tasks succeed
- Use Celery's chord error handling to catch any module failure and surface it in the Pipeline Monitor
- Emit task_progress custom state events from each module task for the "64% COMPLETE" display via SSE

Compliance Note

All Celery task arguments and results pass through Redis. If HIPAA or PII data is ever processed, Redis must be configured with TLS and encryption at rest. Flag this for your DevOps resource now.

4. LLM Orchestration

Options Evaluated

Option	Verdict	Notes
Custom lightweight orchestration (RECOMMENDED approach)	✓ RECOMMENDED	Your PoC is already working Python pipeline scripts. Wrap these with a thin abstraction layer: a PromptTemplate class that loads from DB, an LLMClient class with retry/rate-limit logic, and a ResponseParser per module. This is ~500 lines of code, fully owned, zero vendor lock-in for the orchestration layer.
LangChain	⚠ USE SELECTIVELY	Useful for: the Ask AI Co-Pilot (LLM chains with context injection), prompt template management, and output parsers. NOT recommended as the primary pipeline orchestrator — LangChain adds abstraction overhead on top of Celery, creating two orchestration layers. Use for the interactive Co-Pilot feature only.
LlamaIndex	⚠ FUTURE USE	Excellent for RAG (Retrieval-Augmented Generation) over the Module Knowledge Base PDFs (Section 11, Gap #6). Not needed until the Knowledge Base feature is implemented.
Haystack	✗ NOT RECOMMENDED	Heavier than needed, smaller Python community than LangChain, steeper learning curve for the team.
DSPy	⚠ CONSIDER AT SERIES A	Programmatic prompt optimisation. Relevant when you need to systematically improve confidence scoring accuracy. Not needed at MVP.

Prompt Management

- Store all prompt templates in PostgreSQL (prompts table with version, module_id, tier, content, created_by, is_active)
- Super Admin editable via Settings module — a simple textarea with version history
- Version-controlled: never delete, only deactivate; allows rollback of prompt changes
- Separate templates for Module-level (12 variants) and Enterprise-level (1 template)
- Store template hash with each LLM response — enables cache invalidation when prompt changes

Model Routing Note

The PRD specifies a configurable LLM provider (Anthropic / OpenAI / Gemini). Implement a ModelRouter class in Week 3 that reads from the LLM_PROVIDER environment variable and routes to the correct SDK. This is ~100 lines of code and eliminates provider lock-in at the application level without requiring a heavy orchestration framework.

5. AI/ML Model Layer & LLM API Cost Modelling

Model Options Evaluated

Option	Verdict	Notes
Anthropic Claude 3.5 Sonnet / Claude 3 Haiku	✓ RECOMMENDED PRIMARY	Claude 3.5 Sonnet for enterprise synthesis (Tier 2). Claude 3 Haiku for module-level calls (Tier 1) — 10–20x cheaper per token, sufficient for structured JSON extraction from metric data. Best-in-class for instruction following and structured output, critical for the JSON schema compliance of module reports.
OpenAI GPT-4o / GPT-4o-mini	✓ VIABLE ALTERNATIVE	GPT-4o-mini is comparable to Haiku on cost. GPT-4o comparable to Sonnet on quality. If the team has existing OpenAI tooling, this is an equally valid choice. Slightly higher latency in some regions.
Google Gemini 1.5 Pro / Flash	⚠ COST COMPETITIVE	Gemini Flash is the cheapest option for high-volume module calls. Less mature tooling for structured JSON output. Consider as a cost-optimisation option at Series A scale.
Self-hosted OSS (Llama 3, Mistral)	✗ NOT RECOMMENDED AT MVP	Would require GPU infrastructure (\$500–2,000/month), MLOps expertise the team does not have, and significant fine-tuning work. Not viable within the \$2K/month budget or the 8-week timeline.
AWS Bedrock / Azure OpenAI	⚠ SERIES A	Managed model hosting with VPC isolation and compliance certifications. Necessary for enterprise customers requiring SOC 2 or HIPAA data residency. Not worth the integration complexity at MVP.

LLM API Cost Model — MVP Configuration

Assumptions: 1 enterprise client, 3 pipeline runs per day, 12 module calls per run (Tier 1) + 1 enterprise call (Tier 2). Average tokens per call: Module = 3,000 input + 1,500 output. Enterprise = 18,000 input + 4,000 output.

Call Type	Model	Calls/mo	Cost/call	Monthly Cost
Tier 1 (12 modules × 3 runs × 30 days)	Claude 3 Haiku	1,080 calls	~\$0.005	~\$5.40
Tier 2 (1 enterprise × 3 runs × 30 days)	Claude 3.5 Sonnet	90 calls	~\$0.11	~\$9.90
Ask AI Co-Pilot (est. 200 queries/mo)	Claude 3.5 Sonnet	200 calls	~\$0.06	~\$12.00
TOTAL LLM COST — 1 client				~\$27/month

At 10 enterprise clients (Series A target): ~\$270/month. At 50 clients: ~\$1,350/month. The LLM cost is the most controllable variable — caching identical module inputs eliminates re-runs where data has not changed.

LLM Cost Control Strategies

- Response caching: Cache module-level LLM outputs by (module_id + data_hash). If metric inputs are unchanged, return cached output. Estimated 40–60% call reduction in steady state.
- Tiered model selection: Use Haiku for all Tier 1 module calls; reserve Sonnet for Tier 2 enterprise synthesis and Co-Pilot. Never use Sonnet for structured JSON extraction where Haiku suffices.
- Prompt compression: Trim metric context sent to LLM. Send statistical summaries (mean, std dev, SPC status) rather than raw time-series arrays. Reduces token count by 30–50%.
- Budget alerts: Set hard monthly spend caps via Anthropic/OpenAI API account limits. Alert at 80% of monthly allocation. Hard stop at 100%.
- Batching: For scheduled pipeline runs, batch module calls during off-peak hours to avoid rate limit surcharges.

6. Primary Database

Options Evaluated

Option	Verdict	Notes
PostgreSQL	 STRONGLY RECOMMENDED	Handles the 5-tier hierarchy as an adjacency list or LTREE (ltree extension for path-based queries). JSONB for storing LLM output structured data and dynamic metric attributes. Row-level security for RBAC. Full-text search built in. The Python backend developer almost certainly knows PostgreSQL.
MySQL / MariaDB	 VIABLE	Adequate for the data model but lacks ltree, weaker JSONB support, and no row-level security. Inferior choice for this specific schema.
MongoDB	 NOT RECOMMENDED	Document model is tempting for the hierarchical data, but ACID transactions are essential for the audit trail and RBAC. Mongo's joins for cross-module analytics are painful. Adds a different query paradigm the Python developer may not know.
Supabase (hosted Postgres)	 RECOMMENDED HOSTING	Managed PostgreSQL with built-in auth, row-level security, and a generous free tier (500MB). Reduces DevOps overhead significantly. Direct replacement for raw Postgres at MVP.
PlanetScale	 NOT RECOMMENDED	MySQL-based, lacks PostgreSQL advantages needed here.
CockroachDB	 DEFER	Distributed SQL for global multi-region — relevant at Series A for multi-tenant deployments. Over-engineered for MVP.

Schema Design Principles

- Use an adjacency list model for the 5-tier hierarchy: each node (enterprise, module, submodule, group, metric) has a parent_id. Use PostgreSQL recursive CTEs for tree traversal.
- Add a node_type ENUM column: ENTERPRISE | MODULE | SUBMODULE | GROUP | METRIC. No hardcoded names in application code.
- Store LLM outputs in a separate llm_outputs table with: run_id, tier (MODULE|ENTERPRISE), module_id, prompt_hash, response_json, confidence_score, created_at. This enables cross-period comparison.
- Audit log as an append-only table: audit_events(id, user_id, action, entity_type, entity_id, old_value_json, new_value_json, ip_address, created_at). Never update or delete rows.
- Store prompt templates in prompts(id, module_id NULLABLE, tier, content, version, is_active, created_by, created_at). Load at runtime; cache in Redis with a short TTL.

Compliance Note

PostgreSQL row-level security (RLS) is the cleanest mechanism for future multi-tenant data isolation. Enable it from Day 1 even in single-tenant mode. When V2.0 multi-tenancy is implemented, add tenant_id to all tables and write RLS policies — no application code changes needed.

7. Caching Layer

Options Evaluated

Option	Verdict	Notes
Redis (via Upstash or Railway)	✔ STRONGLY RECOMMENDED	Doubles as Celery broker and application cache — one infrastructure component for two critical functions. Sub-millisecond latency. Python redis-py client is mature. Upstash offers a generous free tier (10,000 commands/day) suitable for MVP.
Memcached	✗ NOT RECOMMENDED	Cannot serve as Celery broker. Two cache layers (Memcached + Redis for Celery) adds operational complexity with no benefit.
In-memory (Python dict)	✗ NOT RECOMMENDED	Cache does not survive process restarts. Incompatible with multiple Celery workers.
DragonflyDB	⚠ FUTURE CONSIDERATION	Redis-compatible, significantly higher throughput. Only relevant if Redis becomes a bottleneck at Series A scale (unlikely below 50 tenants).

Caching Strategy

- LLM module outputs: Cache key = hash(module_id + data_fingerprint). TTL = None (invalidate only on data change). This is the primary cost-saving mechanism.
- Dashboard metric queries: Cache key = hash(enterprise_id + period). TTL = 60 seconds for live dashboards, 300 seconds for historical views.
- Prompt templates: Cache key = prompt_id. TTL = 300 seconds. Avoids DB round-trip on every LLM call.
- User sessions / JWT blacklist: Short TTL (15 minutes). Used for token invalidation on logout.
- Pipeline status: Store real-time pipeline state in Redis (not PostgreSQL) for low-latency SSE updates. Persist final state to PostgreSQL on run completion.

8. Object / Blob Storage

Options Evaluated

Option	Verdict	Notes
Cloudflare R2	 RECOMMENDED	S3-compatible API. Zero egress fees (largest cost saving vs S3). \$0.015/GB storage. Free tier: 10GB + 10M Class A operations/month. Python boto3 works unchanged. Ideal for storing PDF/PPT exports and Module Knowledge Base documents.
AWS S3	 VIABLE	Industry standard. \$0.023/GB + egress fees (\$0.09/GB). Egress adds up when enterprise clients download large PDF/PPT exports. R2 is strictly cheaper for this use case.
Backblaze B2	 VIABLE	Very cheap (\$0.006/GB). S3-compatible. Less polished DX than R2. Good fallback option.
MinIO (self-hosted)	 NOT RECOMMENDED at MVP	Requires managing another service. Operational overhead not justified when R2 free tier covers MVP needs completely.
Supabase Storage	 VIABLE if using Supabase Postgres	Adds storage to the Supabase stack. 1GB free tier. Less performant than R2 for large files.

9. Search & Analytics

Full-Text Search

Option	Verdict	Notes
PostgreSQL full-text search (built-in)	☑ RECOMMENDED at MVP	Sufficient for searching audit logs, LLM report text, and metric names across 1,200 metrics. Zero additional infrastructure. Use tsvector indexes on the relevant columns.
Meilisearch	⚠ INTRODUCE POST-MVP	Excellent DX, very fast, easy to self-host. Relevant when the Ask AI feature needs fast semantic search over historical reports and Knowledge Base PDFs.
Elasticsearch / OpenSearch	✗ DEFER to Series A	Powerful but operationally heavy and expensive. Not justified at MVP with 1–5 enterprise clients.
Algolia	✗ NOT RECOMMENDED	Expensive SaaS. No advantage over Meilisearch for this use case at this budget.

Metric Analytics at Scale

Option	Verdict	Notes
PostgreSQL with proper indexing (MVP)	☑ RECOMMENDED	1,200 metrics × 5 periods per metric = 6,000 rows at MVP. Easily handled by indexed PostgreSQL queries. Composite index on (metric_id, period_id, enterprise_id).
TimescaleDB (Postgres extension)	⚠ INTRODUCE at 10+ clients	TimescaleDB adds automatic time-series partitioning and continuous aggregates to PostgreSQL. Zero application code changes — just add the extension. The natural migration path when metric volume grows.
ClickHouse	⚠ SERIES A	Columnar analytics engine. Relevant when historical metric trend queries across 50+ tenants start impacting OLTP performance. Separate read replica for analytics.
Apache Pinot / Druid	✗ DEFER	Overkill. Real-time OLAP for massive-scale analytics — not needed below 100 tenants.

10. Authentication & Authorization

Authentication Options

Option	Verdict	Notes
Supabase Auth (if using Supabase)	 RECOMMENDED	JWT-based, built-in email/password + social OAuth. Row-level security integrates directly with Supabase Postgres. Free tier covers MVP. SAML/SSO available on paid plan (future enterprise requirement).
Auth0	 VIABLE	Best-in-class for SAML/enterprise SSO. Free tier: 7,500 MAU. Becomes expensive at scale (\$23+/month). Strong compliance features for SOC 2 / HIPAA. Use if enterprise SSO is a near-term requirement.
FastAPI-Users (self-built)	 VIABLE	Keeps auth in-house. More control, more code to maintain. Appropriate if the team wants zero vendor dependency for auth. Adds 1–2 weeks of implementation time.
Clerk	 VIABLE	Excellent DX, generous free tier, built-in UI components. Less enterprise-grade than Auth0 for SAML. Good middle-ground option.
AWS Cognito	 NOT RECOMMENDED	Notoriously difficult DX. Vendor lock-in risk. Free tier is generous but the integration pain is not worth it for a small team.

RBAC Implementation

- Define 4 roles as a PostgreSQL ENUM: SUPER_ADMIN | ADMIN | ANALYST | VIEWER
- Store role assignments in a users_roles table (user_id, role, enterprise_id, granted_by, granted_at)
- Implement role checks as FastAPI dependency injection: def require_role(minimum_role: Role) → current_user decorator
- Enforce at API layer on every endpoint — never trust frontend role checks alone
- Admin nav item: enforce both in React route guard AND in every admin API endpoint
- For future multi-tenancy (V2.0): add tenant_id to role assignments — users can have different roles per tenant

Audit Logging

- Use an append-only PostgreSQL table — never update or delete audit records
- Log every: user login/logout, data read (for sensitive exports), config change, RBAC change, pipeline trigger, export action
- Include: timestamp (UTC), user_id, action_type, entity_type, entity_id, changes_json, ip_address, user_agent
- Index on: user_id, action_type, created_at — for Admin UI filtering
- For SOC 2 readiness: implement immutability at the PostgreSQL level (revoke DELETE/UPDATE on audit_events from the application role)

11. Infrastructure & Hosting

Cloud Provider Options

Option	Verdict	Notes
Railway	☑️ RECOMMENDED for MVP	Deploys Docker containers from GitHub. Managed PostgreSQL and Redis included. Auto-scaling. ~\$20–40/month for MVP workload. Zero Kubernetes config. The fastest path from git push to running service for a 4-person team.
Render	☑️ RECOMMENDED for MVP (alternative)	Very similar to Railway. Slightly better free tier. Managed Postgres (\$7/month), Redis (\$10/month). Background workers (Celery) supported natively. Good DX.
Fly.io	⚠️ VIABLE	Global edge deployment. Better latency for geographically distributed enterprise clients. Slightly steeper DX than Railway/Render. Worth considering if APAC/Middle East clients are a priority from Day 1.
AWS (EC2/ECS/RDS)	⚠️ SERIES A	Full control, compliance certifications (SOC 2, HIPAA Business Associate Agreement). Too much operational overhead for a 4-person team without a dedicated DevOps engineer. Migrate when compliance requirements force it.
GCP / Azure	⚠️ SERIES A	Same rationale as AWS. GCP has an advantage if using Gemini models; Azure if targeting Microsoft enterprise customers.
Hetzner (self-managed VPS)	⚠️ LOWEST COST	Cheapest compute (~\$20/month for a powerful VPS). Requires the team to manage everything. Only viable if a DevOps engineer joins the team. High operational risk for a bootstrapped team.

Containerisation & Orchestration

Option	Verdict	Notes
Docker + Docker Compose (MVP)	☑️ RECOMMENDED	One docker-compose.yml defines: FastAPI, Celery worker, Celery Beat, Redis, PostgreSQL, Nginx. Reproducible across dev, staging, and production. Your Python developer can be productive immediately. No Kubernetes knowledge required.
Kubernetes (K8s)	❌ NOT AT MVP — ⚠️ Series A	Adds significant operational overhead (cluster management, ingress controllers, secrets, service mesh). Not justified for 1–5 enterprise clients. The PRD's K8s mention should be deferred to V1.1. The 8-week timeline does not accommodate a K8s migration.
Docker Swarm	⚠️ VIABLE MIDDLE GROUND	Simpler than K8s but provides basic orchestration. Only relevant if Railway/Render become cost-prohibitive.

Infrastructure as Code

Option	Verdict	Notes
Docker Compose (MVP)	 RECOMMENDED	Single file defines the entire stack. Version-controlled in Git. No additional tooling.
Terraform	 INTRODUCE at Series A	IaC for cloud provider resources. Overkill when using Railway/Render — they handle provisioning. Required when migrating to AWS/GCP/Azure.
Pulumi	 VIALE ALTERNATIVE to Terraform	Same use case as Terraform but uses Python (advantage for this team). Defer to Series A.

12. CI/CD & DevOps

Pipeline Options

Option	Verdict	Notes
GitHub Actions	 STRONGLY RECOMMENDED	2,000 minutes/month free on private repos. YAML-based, huge action ecosystem, zero additional infrastructure. Integrates directly with Railway/Render deployments. The team is already using Git — zero new tooling to learn.
GitLab CI/CD	 VIABLE	Excellent if the team moves to GitLab. Overkill migration if currently on GitHub.
CircleCI / Travis CI	 NOT RECOMMENDED	Paid tiers start immediately. No advantage over GitHub Actions at this budget.
Jenkins	 NOT RECOMMENDED	Self-hosted, requires its own maintenance. Wrong choice for a 4-person bootstrapped team.

Testing Framework (AI Pipeline Platform)

- Backend: pytest + pytest-asyncio. FastAPI TestClient for endpoint testing. Mandatory for all API endpoints before the Week 1 Day 3 contract freeze.
- LLM pipeline tests: Mock the LLM API calls using pytest-mock or respx. Test the chord orchestration logic with mocked module outputs. Never run live LLM calls in CI — cost and flakiness.
- Confidence scoring: Parameterized pytest tests with known metric inputs and expected confidence ranges. Block CI on confidence score regressions.
- Frontend: Vitest (built for Vite) + React Testing Library. Test component rendering and state transitions, not implementation details.
- E2E: Playwright for critical user flows (login, dashboard load, pipeline trigger, PDF export). 3–5 tests only at MVP — full E2E suites are expensive to maintain.
- Contract testing: Use FastAPI's OpenAPI schema as the source of truth. Validate the frozen API contract in CI — any breaking change fails the build.

13. Monitoring, Observability & Alerting

Application Performance Monitoring

Option	Verdict	Notes
Sentry	 STRONGLY RECOMMENDED	Free tier: 5,000 errors/month. Python and React SDKs. Zero-config error capture, performance monitoring, and release tracking. The most important observability tool at MVP — know when things break before the client does.
Datadog	 NOT RECOMMENDED at MVP	Powerful but \$15–30/host/month. Exceeds budget when combined with other infrastructure costs.
New Relic	 NOT RECOMMENDED at MVP	Similar to Datadog in cost and capability. Generous free tier but overkill.
Prometheus + Grafana	 SERIES A	Open-source, self-hosted. Requires a DevOps engineer to maintain. The correct long-term choice when migrating to K8s.

LLM-Specific Observability

Option	Verdict	Notes
Langfuse	 STRONGLY RECOMMENDED	Open-source LLM observability. Traces every LLM call with: prompt, response, latency, tokens, cost, model version. Self-hostable (one Docker container) or cloud-hosted free tier. Critical for debugging confidence score anomalies and prompt regressions. No equivalent tool at this price point.
Helicone	 VIABLE	Proxy-based LLM observability. Free tier covers 10K requests/month. Simpler than Langfuse but less customisable.
LangSmith (LangChain)	 VIABLE if using LangChain heavily	Best observability if using LangChain extensively. Overkill if using custom orchestration.
OpenTelemetry (custom)	 SERIES A	Standardised tracing — implement at Series A to feed into Datadog/Grafana.

Logging & Alerting

- Structured logging: Python structlog library. JSON output. Every log line includes: timestamp, level, service, request_id, user_id (if available), event.
- Railway/Render: Both provide built-in log aggregation at no additional cost. Sufficient for MVP.
- Alerting: Sentry alert rules for error rate spikes. PagerDuty or Slack webhooks for pipeline failures. Uptime Robot (free) for endpoint health checks.
- Celery monitoring: Flower (open-source Celery monitoring dashboard). Deploy as an internal-only service. Shows task queues, worker status, and failed tasks in real time.

14. Full Monthly Cost Estimate

MVP Configuration (1 enterprise client, Team of 4)

Component	MVP Cost/mo	Scale Cost/mo	Notes
Railway / Render (hosting)	\$40–60	\$150–300	FastAPI + Celery worker + Celery Beat + Nginx
Supabase (PostgreSQL)	\$0–25	\$25–100	Free tier covers MVP; Pro at \$25/mo for 8GB DB
Upstash Redis	\$0–10	\$20–50	Free tier: 10K commands/day (Celery broker + cache)
Cloudflare R2 (object storage)	\$0–5	\$5–20	Free 10GB; PDF/PPT exports storage
Anthropic / OpenAI API (LLM calls)	\$25–50	\$200–500	Based on cost model in Section 5; scales with clients
GitHub (private repos)	\$0	\$0–4	Free for teams; Actions 2K min/mo free
Sentry (error monitoring)	\$0	\$26	Free 5K errors/mo; Team plan \$26/mo when needed
Langfuse (LLM observability)	\$0	\$0–30	Self-hosted free; cloud tier for team convenience
Domain + SSL	\$10–15	\$10–15	Annual domain ~\$10–15; SSL via Let's Encrypt (free)
Uptime Robot (monitoring)	\$0	\$0–7	Free 50 monitors; Pro for SMS alerts
TOTAL ESTIMATED MONTHLY	~\$75–165/mo	~\$430–1,030/mo	Well within \$2K budget

Budget Headroom

At MVP with 1 client, total infrastructure cost is approximately \$75–165/month. The \$2,000/month budget ceiling allows for 10–12x growth in client load, LLM API costs, and team tooling before requiring a hosting architecture change. The model is economically sound.

15. Unresolved Engineering Gaps — Decision Register

Ten UI elements are visible in the design artefacts with no corresponding Functional Requirements. Each must be explicitly decided before or during Sprint 1. The table below classifies each gap and provides an architectural recommendation.

Gap / Element	Architectural Decision	MVP Action	Rationale
EMERGENCY OVERRIDE	Requires a dedicated API endpoint: POST /pipelines/emergency-stop. Needs SuperAdmin role check, audit log entry, and a mechanism to signal all running Celery tasks to gracefully terminate (revoke with terminate=True).	Stub	Stub the button with a modal: "This feature is being configured. Contact Super Admin." Implement the kill-switch logic in Sprint 3. Do NOT leave it wired to nothing in production.
Rollback DomainLast	Rollback = restore previous run's LLM output and score for a specific module. Requires a pipeline_runs table with a is_current flag. Rollback = flip the flag. NOT a DB rollback.	Design Now	The data model decision (is_current flag on pipeline_runs) must be made before the DB schema is finalised in Week 1. The UI action can be implemented in Sprint 3.
Alert Schema (Levels 1–4)Severity	Define a severity ENUM: INFO LOW MEDIUM HIGH CRITICAL. Map to numeric levels for API compatibility. Store per-metric and per-run in an alerts table.	Design Now	Blocks DB schema and all alert-related API endpoints. Must be resolved at the schema design stage in Week 1 — cannot be deferred without creating technical debt in Sprint 1.
FORCE RECALC	Triggers recalculation for a single sub-module/metric without running the full pipeline. Requires a targeted Celery task: recalc_submodule(submodule_id, run_id). Admin-only.	Stub	Stub in MVP (button visible, fires a notification: "Recalculation queued"). Full implementation in Sprint 4. Low demo impact.
Deploy New Watcher	Most complex unspecced item. Implies dynamic metric registration — a new metric watcher can be added to a module at runtime. Requires a watchers table and a dynamic pipeline loader.	Defer	Too complex for MVP. Defer to V1.1. Stub the button as "Available in Enterprise tier." This is a product feature, not just a UI fix.
Module Knowledge Base	User-uploaded PDFs stored in R2. Metadata (filename, module_id, uploaded_by) in PostgreSQL. Used as RAG context for Ask AI. Requires LlamaIndex or similar for chunking and embedding.	Stub	Stub: store files in R2, show links in UI, but do NOT implement RAG at MVP. The full RAG implementation is a Sprint 5–6 item if timeline permits; otherwise V1.1.
Report ID Format (VL-AI-992-DELTA)	Standardise format: {PREFIX}-{ENTERPRISE_ID}-{RUN_SEQUENCE}-{MODULE_HASH_SHORT}. Generate at report creation time. Store in llm_outputs table.	Design Now	Must be in the DB schema from Day 1. The format is trivial to implement (a few lines in the report creation service) but the column must exist before Sprint 2 API work.
Heatmap Navigation	Is the heatmap a tab/toggle within the Module Dashboard or a separate route? Recommendation: make it a toggle tab within the Module Dashboard (saves one route, keeps navigation simple at MVP).	Design Now	Must be resolved before the API contract freeze on Day 3 — it determines whether a separate API endpoint for heatmap data is needed or if it reuses the module dashboard endpoint.

Gap / Element	Architectural Decision	MVP Action	Rationale
Processing Rate Metric (12.4 GB/s)	If real: add a pipeline_metrics table logging data_volume_bytes and duration_ms per run. Calculate GB/s at query time. If decorative at MVP: render a static/simulated value with a clear "sample data" label.	Stub	Stub with simulated data at MVP. Implement real instrumentation in Sprint 4 when pipeline monitoring APIs are built. Label it clearly in the UI as estimated.
SSO Placeholder (V1.0)	A Settings page field that stores an IDP metadata URL but does not activate SAML. Wire a disabled "Connect SSO" button that shows "Available on Enterprise plan."	Stub	One settings form field + one disabled button. 30 minutes of work. Defer actual SAML integration to V1.1 when Auth0 or Supabase SSO is configured.

16. API Contract Freeze — Week 1 Day 3

The Week 1 Day 3 API contract freeze is a hard deadline: from that point, the frontend engineer builds against mocks and cannot be blocked by unresolved backend decisions. However, two critical open questions remain unresolved — Q3 (module hierarchy) and Q6 (confidence scoring). Here is how to proceed:

How to Freeze a Contract When Q3 Is Unresolved

- Use a placeholder module list of 12 placeholder IDs (module_001 through module_012) in the schema and API responses. The frontend builds against these placeholders.
- Define the MODULE shape completely: { id, name, score, status, sub_modules[], metrics[], ai_output, confidence }. The shape is stable even if the names are placeholders.
- When the real module list is confirmed (must be by Week 1 Day 3 — it is a blocking decision), run a migration to populate the real names. No API contract change required.

How to Freeze a Contract When Q6 Is Unresolved

- The confidence score is a float 0.0–1.0 (or 0–100%) in the API response. The field shape is stable regardless of the underlying statistical model.
- At MVP, implement a simple confidence heuristic: weighted average of metric variance scores within the module. Return this as the confidence value.
- When the real statistical model is defined (Week 3 target per the SWOT analysis), it replaces the heuristic calculation internally. The API contract does not change.

Critical Pre-Freeze Actions

The following MUST be resolved before Day 3 to unblock the API contract: (1) Confirm exact list of 12 modules — even placeholder names. (2) Resolve heatmap navigation (tab vs route). (3) Resolve alert severity schema (1–4 levels). (4) Add SSE endpoints for pipeline progress and Co-Pilot streaming to the contract. (5) Confirm deployment target (Cloud SaaS) to finalize environment variable and secrets strategy.

17. Phased Adoption Path

MVP vs Series A vs Defer

Component / Decision	Phase	Rationale
React + Vite + Zustand + Tailwind	MVP	Already decided; IIT engineer productive from Day 1
shadcn/ui + Recharts	MVP	Composable with Aegis Quantum tokens; no design system conflicts
FastAPI + Pydantic	MVP	Native async; auto OpenAPI docs; Python team productive
REST API (JSON)	MVP	Simple, debuggable, matches team skill; no GraphQL overhead
SSE for pipeline status + Co-Pilot	MVP	Required for live progress display; FastAPI native; Day 3 contract item
Celery + Redis (chord pattern)	MVP	Exact match for 12→1 barrier requirement; PoC already uses Python
PostgreSQL (Supabase hosted)	MVP	JSONB + LTREE + RLS; zero DevOps; free tier covers launch
Redis (Upstash)	MVP	Free tier covers MVP; doubles as Celery broker and cache
Cloudflare R2	MVP	Zero egress fees; S3-compatible; free 10GB covers MVP exports
Anthropic Claude 3 Haiku (Tier 1)	MVP	Cost-optimised module calls; ~\$5/month for 1 client
Anthropic Claude 3.5 Sonnet (Tier 2)	MVP	Enterprise synthesis; quality justified for board-level outputs
Custom ModelRouter class	MVP	Provider-agnostic; ~100 lines; no LangChain overhead for pipelines
LangChain (Co-Pilot only)	MVP	Use for Ask AI chat chains and context injection only; not pipeline orchestration
Railway or Render (hosting)	MVP	Docker-native PaaS; \$40–60/month; zero K8s ops burden
Docker + Docker Compose	MVP	Single docker-compose.yml; reproducible; team knows Docker
GitHub Actions (CI/CD)	MVP	Free 2K minutes/month; integrates with Railway/Render deploy hooks
PostgreSQL full-text search	MVP	Built-in; sufficient for 1,200 metrics + audit log search
Sentry (error monitoring)	MVP	Free 5K errors/month; most important observability tool at launch
Langfuse (LLM observability)	MVP	Self-hosted free; essential for debugging confidence score issues
Supabase Auth or Auth0	MVP	JWT + RBAC; SAML placeholder only; enterprise SSO in V1.1
PostgreSQL RLS (multi-tenant prep)	MVP	Enable from Day 1; free; V2.0 multi-tenancy requires it
MSW (Mock Service Worker)	MVP	Frontend builds against mocks from Day 3; unblocks parallel development
Nginx reverse proxy	MVP	SSL termination + static files; zero cost; industry standard

Component / Decision	Phase	Rationale
TimescaleDB (time-series extension)	Series A	Add when metric volume or query latency becomes noticeable (10+ clients)
Kubernetes (K8s)	Series A	Introduce with dedicated DevOps engineer; not viable at 4-person team
AWS/GCP/Azure migration	Series A	Required for SOC 2 compliance certifications and enterprise SLAs
AWS Bedrock / Azure OpenAI	Series A	VPC-isolated LLM calls; required for HIPAA/finance-grade customers
Prometheus + Grafana	Series A	Self-hosted APM; introduce with K8s migration
GraphQL API layer	Defer	No mobile app, no third-party integrations at MVP — not justified
Kafka (event streaming)	Defer	Real-time ingestion is V2.0 scope; batch pipeline is V1.0
Deploy New Watcher feature	Defer	Complex dynamic pipeline registration; not needed for demo
RAG over Module Knowledge Base	Defer	LlamaIndex + embeddings; defer until Ask AI Phase 2
ClickHouse (analytics)	Defer	Columnar analytics for 100+ tenants; not needed below Series A scale
Self-hosted OSS LLMs	Defer	GPU infra + MLOps burden; not viable at this budget or team size

18. Final Ranked Stack Recommendation

The following is the single most technically sound, cost-effective, and scalable combination for TBD2, with explicit rationale for why each choice beats its alternatives given the constraints.

#	Layer	Choice	Why This Beats Alternatives
1	Frontend	React + Vite	Auth-gated dashboard; IIT engineer productive Day 1; Aegis Quantum Tailwind tokens drop in directly; no SSR overhead.
2	State	Zustand	Global Co-Pilot panel + pipeline status state; minimal boilerplate; 15-minute learning curve for any React developer.
3	Components	shadcn/ui + Recharts	Headless + Tailwind-composable; no design system conflicts; Recharts for SPC charts; D3 only where Recharts falls short.
4	Backend	FastAPI + Pydantic	Native async for 12 parallel LLM calls; auto OpenAPI docs enable the Day 3 contract freeze; type safety catches bugs early.
5	Real-Time	SSE (FastAPI native)	One-directional push for pipeline progress and Co-Pilot streaming; zero additional infrastructure; WebSockets not needed at MVP.
6	Task Queue	Celery + Redis	chord primitive = the 12→1 barrier pattern with zero custom coordination code; PoC already in Python; Celery Beat for scheduling.
7	LLM Orchestration	Custom + LangChain (CoP)	Custom ModelRouter for pipelines (zero framework lock-in); LangChain only for Ask AI Co-Pilot chains; 500 lines of owned code.
8	LLM Models	Anthropic (Haiku + Sonnet)	Haiku for 12 module calls (~\$5/mo/client); Sonnet for enterprise synthesis; best structured JSON output; model-agnostic router allows switching.
9	Primary DB	PostgreSQL (Supabase)	LTREE hierarchy; JSONB for LLM outputs; RLS for future multi-tenancy; managed hosting removes DevOps overhead at MVP.
10	Cache / Broker	Redis (Upstash)	One service for two critical functions: Celery broker + LLM response cache. Free tier covers MVP entirely.
11	Object Storage	Cloudflare R2	Zero egress fees vs S3 (saves \$50+/month at scale); S3-compatible API; boto3 works unchanged; free 10GB covers all MVP exports.
12	Auth / RBAC	Supabase Auth	Integrates with Supabase Postgres RLS; JWT built-in; SAML placeholder for V1.1; saves 1–2 weeks vs building auth from scratch.
13	Hosting	Railway	Docker-native PaaS; one-command deploys from GitHub; managed Postgres + Redis available; \$40–60/month; zero K8s burden.
14	CI/CD	GitHub Actions	Free 2K min/month; YAML pipelines; deploy hooks for Railway/Render; entire team already uses GitHub.

#	Layer	Choice	Why This Beats Alternatives
15	Error Monitoring	Sentry	Free 5K errors/month; Python + React SDKs; zero config; most critical observability tool for a 4-person team.
16	LLM Observability	Langfuse	Only tool purpose-built for LLM trace/cost/prompt debugging at this price point; self-hosted = zero cost; essential for confidence score debugging.

Why This Stack Beats All Alternatives

The entire stack fits inside \$165/month at MVP, leaving \$1,835 of the \$2,000 budget ceiling as headroom. Every component is Python-or-TypeScript-native — no language context switching. Zero components require a dedicated DevOps engineer to operate. Every MVP choice has a clear, non-breaking upgrade path to the Series A configuration. The stack has been validated against the specific constraints of this platform: the Celery chord covers the 12→1 barrier, PostgreSQL LTREE handles the 5-tier hierarchy without a graph database, SSE covers the real-time gap identified in the PRD analysis, and Langfuse covers the LLM observability gap that no other tool addresses at zero cost.

Three Decisions to Make Before Writing One Line of Code

Priority	Decision	Deadline	What it Unblocks
● P0	Confirm the exact list of 12 Modules (Q3) — even placeholder names	Week 1 Day 1	DB schema, pipeline wrappers, all backend Sprint 1 work
● P0	Confirm deployment target: Cloud SaaS (Railway/Render) or on-premise	Week 1 Day 1	Infrastructure setup, secrets management, Docker Compose vs K8s decision
● P0	Confirm primary LLM provider (Anthropic recommended, OpenAI viable)	Week 1 Day 1	LLM API wrapper, prompt engineering, cost modelling, API key management
● P1	Resolve heatmap: tab within Module Dashboard vs separate route	Week 1 Day 3	API contract freeze — determines if a separate heatmap endpoint is needed
● P1	Define alert severity schema (Level 1–4 numeric model)	Week 1 Day 3	DB schema alerts table, all alert-related API endpoints
● P2	Define confidence scoring methodology (Q6)	Week 3	Confidence engine implementation in Sprint 3; MVP heuristic bridges the gap

END OF DOCUMENT